

Homework 2

Jamie Meng

May 2, 2026

1 Problem 1

When we are alternating between left and right replacements, we are move along a path in a graph. And we need to visit all of them at once. Therefore, they should be a Euler circuit. Because we need to find 2 differnt orders, the circuit is in a graph $G = (V, E)$ with vertex set $V = X \uplus Y$ where $X := \{x_1, \dots, x_{11}\}$ and $Y := \{y_1, \dots, y_{11}\}$, and edge set $E = \{x_i, y_j\} : 1 \leq i, j \leq 11, i \neq j$.

It is a complete graph, and each vertices' degree is 10 which is even. So there exists a Euler circuit. Therefore, there is a sequence of 109 replacements.

2 Problem 2

(a)

Proof by Induction: Basecase: If $n=1$, there is no edge. Therefore, it's less than dn .

Induction Hypothesis: for any n -vertex d -degenerate graph G contains at most $e(G) \leq dn$ edges.

Induction step: for a $n+1$ vertex d -degenerate graph G , there exist a vertex with less than or equal to d neighbors. We can remove that vertex and all neighbors of it. And we get a induced subgraph G' and it is still a d -degenerate graph. So G' is a n vertex d -degenerate graph. It's edges number is less than dn . When we add back the removed vertex. G 's edge number is less than $dn+d$, which is less than $d(n+1)$. And we are done.

(b)

Proof by Induction: Basecase: If $n=1$, we can color it with one color.

Induction Hypothesis: for any n -vertex d -degenerate graph G can be colored with $d+1$ colors.

Induction step: for a $n+1$ vertex d -degenerate graph G , there exist a vertex with less than or equal to d neighbors. We can remove that vertex and all neighbors of it. And we get a induced subgraph G' and it is still a d -degenerate graph. So G' is a n vertex d -degenerate graph and it can be colored with $d+1$ color. When we add back the removed vertex. Becasue the vertices only have less than or equal to d neighbors. Then, we can also color it with $d+1$ colors. And we are done.

(c)

Claim: the graph G is $(l-1)$ -degenerate.

Proof: for all induced subgraph in G , the longest path is shorter than or equal to the longest path in G . Consider the end point of each path. It can only have at most $(l-1)$ neighbors which are all in the path. Because if one is not in the path, the path is not the longest one. Therefore, $\delta(H) \leq l-1$. And the graph G is $(l-1)$ -degenerate.

Then $\chi(G) \leq l$

3 Problem 3

(a)

The number of color we need to use is the maximum color number we use. Then, we need to find the maximum color number we use. For each vertex, we need to color it with smallest number. If it has d_i neighbors, the worst case is they are all colored with different colors, then the color number should be $d_i + 1$. But if $d_i + 1 \leq i$, then we just need to number it i since there are only at most $(i-1)$ number of color used. Then,

$$\chi(G) \leq \max_{1 \leq i \leq n} \min\{d_i + 1, i\}$$
$$\chi(G) \leq 1 + \max_{1 \leq i \leq n} \min\{d_i, i - 1\}$$

(b)

If we know how to color it with the least number say k . Then, some of them is colored with $1, 2, 3, \dots, k$. Then, we can make the algorithm color 1 first. And then 2, to k . For each coloring, if it is colored with s by coloring it with the least number. Our algorithm must color is with number less than or equal to s . Therefore, we can color it optimally.

(c)

For a path with more than 4 vertices. We can just see the first 4 vertices. If the first 4 need to use 3 colors, then the whole one need 4 colors.

When we first color first vertex with color 1 and then color forth vertex with color 1 because its neighbors do not have color yet, then if we thirdly color the second vertex with 2 because the first vertex is 1, we can only color the third vertex with color 3, because its neighbors are already colored with 1 and 2. Then we are done.

(d)

Consider the bipartite graph G_n with the two vertex-classes $A_n = \{a_1, \dots, a_n\}$ and $B_n = \{b_1, \dots, b_n\}$, where there is an edge between vertex $a_i \in A_n$ and vertex $b_j \in B_n$ only if $i \neq j$. It is a bipartite graph because we can color all vertices in A_n be color 1 and B_n be color 2. If our algorithm start in a_1 and to b_1 then a_2 then b_2 Then a_1 and b_1 will be 1. And a_i and b_i will be i . Because when we

coloring a_i and b_i , all of their neighbors are already colored with 1 to $i-1$. Finally, it will need n colors.